

Entornos de Desarrollo.

Tarea unidad 1.

Parte 1: Investigación.....	2
Modelo en Cascada.....	2
Análisis:.....	2
Diseño:.....	3
Implementación:.....	3
Verificación:.....	3
Pruebas unitarias:.....	4
Pruebas de integración:.....	4
Mantenimiento:.....	4
Scrum.....	5
Roles en Scrum.....	5
Product Owner.....	6
Scrum Master:.....	6
Equipo de Desarrollo:.....	6
Artefactos de Scrum.....	6
Product Backlog:.....	6
Sprint Backlog:.....	7
Incremento:.....	7
Eventos de Scrum.....	7
El Sprint o interacción:.....	7
Sprint Planning:.....	7
Daily Scrum:.....	8
Sprint Review:.....	8
Sprint Retrospective:.....	8
Ventajas y desventajas de cada modelo.....	8
Ejemplo práctico.....	10
Parte 2: Afianzamiento de conocimientos.....	11
Fase de Análisis, explicación.....	11
Ejemplo práctico.....	11
Requisito 1: Gestión de Usuarios y Control de Acceso Basado en Roles (Funcional).....	12
Requisito 2: Gestión de Préstamos y Devoluciones con Notificaciones Automáticas (Funcional).....	12
Requisito 3: Sistema de Búsqueda y Disponibilidad de Libros Accesible en Línea (Funcional y No Funcional).....	13
Anexo.....	14
Prueba de trabajo personal:.....	14

Autor: Juan Cruz Garrido Suso

Fecha 29/10/2025

Parte 1: Investigación

Para esta tarea escojo el modelo en cascada y Scrum.

Modelo en Cascada

El modelo en cascada consiste en la planificación detallada de modo lineal y secuencial de un proyecto, en el que cada fase se va desarrollando secuencial y dependientemente, de modo que el siguiente se apoyará en los anteriores y éstos no se deben modificar una vez concluidos.

Consiste en cinco fases: Análisis, Diseño, Implementación, Verificación y Mantenimiento.

Análisis:

En la fase de análisis se recopilan y documentan los requisitos, el producto final de esta fase es un documento ERS (Especificación de Requisitos de Software) que contendrá todos los requisitos recopilados con orden de prioridad.

Los requisitos se dividen en dos categorías, los requisitos funcionales y los no funcionales. Los funcionales vienen a decirnos el qué debe hacer el sistema, mientras que los no funcionales vienen a detallar el cómo lo deben hacer. Por ejemplo, funcionalmente una aplicación permite el préstamo de libros y gestionar los retrasos en las devoluciones, mientras que un requisito no funcional podría ser que el sistema debe ser capaz de gestionar 10.000 usuarios simultáneos sin degradación del rendimiento, o que cualquier consulta al catálogo debe devolver resultados en menos de 2 segundos.

Cualquier asunto que no quede clara y correctamente descrito en esta fase podrá generar grandes dificultades en el desarrollo del proyecto, ya que todo el trabajo a realizar se derivará directa o indirectamente del resultado de esta fase.

Diseño:

La siguiente fase del modelo en cascada es el diseño, en esta fase se definirán:

- **Diseño de la Información:** El modelo de datos, cómo representamos y estructuramos la información que gestionará nuestra aplicación.
- **Diseño de la interacción:** La interfaz de los diferentes tipos de usuarios, cómo se usa y qué información y acciones disponibles tiene un tipo de usuario dado en un momento dado.
- **Diseño del funcionamiento:** Qué se debe hacer con la información, cómo procesarla y almacenarla para cumplir con los propósitos definidos en la fase anterior.

De esta manera, el fruto de esta fase debe perfilar perfecta y detalladamente el uso de la aplicación y qué hace ésta en cada escenario posible, es el paso previo a la codificación y por tanto debe incluir todos los posibles detalles antes de pasar a la construcción.

Implementación:

La fase de implementación es la fase en la que de hecho se construye (programa) la aplicación, es altamente dependiente de disponer un buen diseño, lo que a su vez es altamente dependiente de disponer de una buena lista de requisitos bien detallados y definidos en la fase de análisis

El código debe estar bien organizado, ser correcto, fácil de leer y entender, eficiente y portable.

Verificación:

En esta fase se comprueba que la aplicación resuelve, de hecho, todos los requisitos, no genera errores y se comporta correctamente en todos los escenarios.

Usualmente se generan datos de pruebas en una bbdd específica para pruebas que permitan aislar un estado fijado de los datos y generar pruebas sobre ellos, las pruebas pueden ser de dos tipos, pruebas unitarias y pruebas de integración.

Pruebas unitarias:

Las pruebas unitarias reciben su nombre de que se comprueba el buen funcionamiento de una parte del software, hay herramientas para lanzar conjuntos de estas pruebas y generar un informe que servirá para identificar errores ya que al disponer de unos datos fijados pueden definirse pruebas específicas para comprobar lo que nuestra aplicación hace con ellos sabiendo de antemano el resultado que deberá ofrecer y por tanto comprobando su buen funcionamiento en la medida que la calidad de estas pruebas ofrezca.

Pruebas de integración:

Las pruebas de integración comprueban el funcionamiento completo del sistema, usando las interfaces construidas, con objeto de poder entregar la aplicación al usuario final. Se recomienda acercarse todo lo posible al escenario real de uso de la aplicación desde el punto de vista de la infraestructura, información, tipos de usuarios y procesos corrientes en los que deberá cumplir su función, para ello pueden dedicarse perfiles de beta testers.

Tras finalizar esta fase con éxito, el proyecto se entrega al usuario o cliente.

Esta fase culmina con las Pruebas de Aceptación de Usuario (UAT), donde perfiles como los beta testers o el propio cliente validan que la aplicación cumple con lo esperado antes de la entrega final.

Mantenimiento:

La fase de mantenimiento es aquella en la que el software, una vez en explotación, recibe cambios en función de diferentes escenarios que se pueden presentar.

Se pueden describir los siguientes tipos de mantenimiento:

- **Perfectivos:** Para mejorar la funcionalidad del software.
- **Evolutivos:** El cliente tendrá en el futuro nuevas necesidades. Por tanto, serán necesarias modificaciones, expansiones o eliminaciones de código.
- **Adaptativos:** Modificaciones, actualizaciones... para adaptarse a las nuevas tendencias del mercado, a nuevos componentes hardware, nuevas condiciones especificadas por organismos reguladores, etc.
- **Correctivos:** La aplicación tendrá errores en el futuro.

Scrum

Scrum es una metodología de desarrollo englobada en las denominadas metodologías ágiles (agile). Propone una aproximación opuesta en algunas características a lo propuesto por el modelo en cascada, incidiendo en la flexibilidad y entrega rápida de valor al cliente, exige una comunicación con el usuario/cliente mucho más frecuente y persistente.

Esta metodología se fundamenta en un proceso de control empírico, basado en la transparencia, la inspección y la adaptación para gestionar la incertidumbre y los problemas complejos, en lugar de una planificación inicial detallada, las decisiones se toman a partir de la observación y la experimentación durante el desarrollo, incluyendo al usuario o cliente en el proceso incremental, de modo que se maximice el valor entregado y la flexibilidad.

Scrum se estructura en torno a un conjunto de roles, artefactos y eventos que organizan el trabajo en ciclos cortos llamados Sprints.

Roles en Scrum

Un equipo Scrum está compuesto por tres roles definidos que interaccionan según sus papeles:

Product Owner

Actúa como representante de los clientes y usuarios, estableciendo y priorizando los elementos de la lista de requisitos del producto (Product Backlog).

Scrum Master:

Actúa como un líder que gestiona el proceso, ayuda a eliminar problemas que puedan obstaculizar el progreso del equipo y organiza y facilita las reuniones. Su papel es resolver obstáculos.

Equipo de Desarrollo:

Está formado por los profesionales encargados de desarrollar y entregar un incremento de producto al final de cada Sprint.

Son equipos autoorganizados y multifuncionales, con la responsabilidad de planificar su trabajo y completarlo.

Artefactos de Scrum

Los artefactos de Scrum son herramientas o documentos que proporcionan transparencia y estado sobre el trabajo, el progreso y los objetivos.

Los tres artefactos principales son:

Product Backlog:

Es una lista dinámica y priorizada que contiene todos los requisitos, funcionalidades, mejoras y correcciones necesarias para el producto. Es gestionada exclusivamente por el Product Owner y es la única fuente de trabajo para el equipo.

Sprint Backlog:

Es un subconjunto de elementos del Product Backlog que el equipo de desarrollo prevé completar durante un Sprint. También incluye el plan para entregar el incremento del producto.

Incremento:

Es el resultado tangible del trabajo realizado durante un Sprint. Consiste en la suma de todos los elementos del Product Backlog completados durante el Sprint actual y los Sprints anteriores, y debe ser una versión del producto que sea potencialmente entregable.

Eventos de Scrum

Scrum propone una serie de eventos para crear regularidad y minimizar la necesidad de reuniones no definidas en el marco. Todos los eventos tienen una duración máxima establecida.

El Sprint o interacción:

Representan los ciclos de desarrollo y entrega de partes usables y con valor para el cliente, un bloque de tiempo de duración fija (generalmente entre una y cuatro semanas) durante el cual se crea un incremento de producto "Terminado", utilizable y potencialmente entregable.

Un nuevo Sprint comienza inmediatamente después de la conclusión del anterior.

Sprint Planning:

Se realiza al inicio de cada Sprint para planificar el trabajo que se va a realizar. Todo el equipo Scrum, especialmente el product owner, colabora para definir un objetivo para el Sprint (Sprint Goal) y seleccionar los elementos del Product Backlog que se abordarán. Con la lista de funcionalidades extraída del BackLog que se haya

realizado se planifican los detalles técnicos para su consecución con éxito durante el Sprint.

Daily Scrum:

Es una reunión diaria de pocos minutos para el equipo de desarrollo. Su objetivo es sincronizar actividades e identificar impedimentos. Típicamente, cada miembro responde a tres preguntas: ¿Qué hice ayer?, ¿Qué haré hoy? y ¿Qué impedimentos tengo?

Sprint Review:

Se celebra al final del Sprint para inspeccionar el incremento y adaptar el Product Backlog si es necesario. El equipo Scrum presenta el trabajo completado a las partes interesadas para recibir sus comentarios.

Sprint Retrospective:

Ocurre después de la Sprint Review y antes de la siguiente Sprint Planning. Su propósito es que el equipo se inspeccione a sí mismo e identifique mejoras en sus procesos para el próximo Sprint.

Ventajas y desventajas de cada modelo

La diferencia esencial entre estos dos modelos es la flexibilidad, el modelo en cascada está fijado, con fases que quedan comprometidas, y con poca o ninguna interacción con el cliente/usuario en largos periodos de tiempo. Así, el producto final se va perfilando en función de los requisitos y decisiones tomadas en el pasado sin posibilidad de cambiarlos. Esto ofrece la ventaja de conocer a largo plazo, pero expone riesgos serios si estas decisiones no fueron las correctas.

Sin embargo los modelos ágiles y Scrum en particular, proponen una aproximación flexible, adaptable y mucho más integrada con el usuario/cliente, lo que permite ofrecer valor y satisfacción más continuamente e integrar al usuario en la decisiones, a costa de perder la perspectiva a largo plazo.

Así, para cada metodología:

Modelo en cascada

Ventajas

- Fácil de seguir
- Control claro
- Proyectos con reglas fijas

Desventajas

- Poca flexibilidad
- Los errores tardan en aparecer y pueden implicar un impacto muy importante
- El cliente espera sin intervención y con pocas posibilidades de probar elementos del producto final.

Scrum

Ventajas

- Alta flexibilidad y adaptación a los cambios
- Entregas parciales y rápidas de producto funcional al cliente/usuario
- Fomenta la colaboración, la transparencia y la motivación del equipo

Desventajas

- Difícil definir plazos y costes fijos al inicio del proyecto
- Requiere perfiles experimentados, fluidez técnica y personal.
- Presenta dificultades de escalamiento, aunque son solventables, puede implicar esfuerzos importantes de coordinación

NOTA: Me he documentado en internet sobre algunas de estas consideraciones, especialmente en Scrum.

Ejemplo práctico

Para mostrar un ejemplo de uso de cada metodología, y siguiendo el ejemplo del apartado siguiente, voy a escoger un proyecto de bibliotecas para una comunidad autónoma.

La primera gran fase de este proyecto puede realizarse con metodología de cascada, ya que los requisitos pueden ser claramente definidos desde el principio y permitirá una planificación de tiempos y costes a largo plazo muy eficaz. Aquí se incluye el desarrollo básico de la funcionalidad principal y la integración con los sistemas de autenticación disponibles, por ejemplo.

Una vez entregada esta primera versión, y ya en fase de mantenimiento, se podría escoger un equipo y con él hacer desarrollo con metodología ágil, Scrum por ejemplo, con objeto de hacer un mantenimiento evolutivo y si es necesario, correctivo.

Así se podría recoger la opinión de trabajadores y usuarios sobre qué características de la aplicación pueden mejorarse, ya que esta metodología está enfocada en la flexibilidad y todos podrán observar esos cambios con rapidez y certeza. En esta fase podría realizarse tareas de retoques de interfaz, o inclusión de actividades diferentes al préstamos de libros pero realizadas por las bibliotecas, como cursos, préstamos de instalaciones, actividades sociales de todo tipo etc.

Parte 2: Afianzamiento de conocimientos

Para esta parte escojo el análisis.

Fase de Análisis, explicación

Como ya se ha explicado anteriormente para el modelo de cascada, esta fase es esencial, especialmente cuantos más elementos y con mayor alcance pretenda recoger.

Esta fase consiste en recopilar y describir con suficiente precisión todos los requisitos que se esperan del software que se quiere desarrollar. Se dividen en funcionales y no funcionales, siendo los primeros los más importantes, ya que describen el qué se espera que la aplicación haga. Los no funcionales, si bien también pueden ser muy importantes, se refieren más bien al cómo se hacen ciertas cosas, por ejemplo, coloridos de interfaz, o tiempo de espera a una solicitud, o cuestiones de escala como número de libros o concurrencia de usuarios que se debe tolerar.

En cualquier caso es una fase crítica, la razón es que al ser la raíz del proyecto, si éste es a largo plazo y requiere muchos recursos, nos exponemos a un riesgo muy alto si no se realiza correctamente. Es de esperar que, en este caso, esta fase sea considerada la más delicada y, personalmente espero, que incluya a todos los perfiles necesarios y con la dedicación precisa, incluyendo al cliente/usuario.

Ejemplo práctico

Dado el desarrollo de una aplicación de gestión para una biblioteca, redacto aquí tres posibles requisitos que se podrían recoger en su fase de análisis.

Requisito 1: Gestión de Usuarios y Control de Acceso Basado en Roles (Funcional)

El sistema debe permitir la creación y gestión de diferentes tipos de usuarios, principalmente un administrador y bibliotecarios, además de usuarios, que a su vez, podrán ser estudiantes o profesores.

El rol de administrador puede ver y gestionar todo, es el administrador y su perfil es de administración del sistema. Tiene acceso a todo, específicamente a las acciones de los demás usuarios en el sistema.

El rol de bibliotecario debe tener permisos para añadir y modificar registros de libros, préstamos y usuarios, así como para generar informes de préstamos y estadísticas.

El rol de usuario solo debe tener permisos para buscar en el catálogo, ver su historial de préstamos y realizar reservas de libros en caso de ser profesor.

Requisito 2: Gestión de Préstamos y Devoluciones con Notificaciones Automáticas (Funcional)

El sistema debe permitir a los bibliotecarios registrar el préstamo de libros a los usuarios (estudiantes y profesores) a través de un escáner de código de barras o mediante la introducción manual del ISBN o identificador del libro.

Igualmente, debe permitir registrar las devoluciones. El sistema deberá enviar automáticamente notificaciones por correo electrónico a los usuarios tres días antes de la fecha de vencimiento del préstamo y también el día del vencimiento si el libro no ha sido devuelto.

Requisito 3: Sistema de Búsqueda y Disponibilidad de Libros Accesible en Línea (Funcional y No Funcional)

El sistema debe ofrecer un catálogo en línea que permita a los usuarios buscar libros por título, autor, género o palabra clave desde cualquier dispositivo con acceso a internet.

Los resultados de la búsqueda deben mostrar la portada del libro, su resumen, ubicación en la estantería y su estado de disponibilidad en tiempo real (disponible, prestado, reservado).

La consulta de búsqueda en el catálogo debe ejecutarse y mostrar los resultados en menos de 3 segundos para garantizar una buena experiencia de usuario. (No funcional)

Comentario:

Quizá algunos de estos requisitos abarquen muchas cosas, pero lo incluyo así ya que considero que son soluciones estándar (uso de perfiles, por ejemplo) y lo esperado de la aplicación en tanto a lo que tratan de describir es fácilmente comprensible dada la naturaleza bien conocida de lo que se espera de una aplicación así.

Anexo

Prueba de trabajo personal:

